

Timing-Driven Logic Remapping with Local Physical Context

Zijian Jiang, Hongyang Pan, Cunqing Lan, Keren Zhu*
State Key Laboratory of ASIC and System, Fudan University, Shanghai, China

Abstract—The timing behavior of a mapped circuit depends on both its logic implementation and the physical environment in which that implementation is realized. Revisiting mapping decisions after placement therefore requires a search procedure that accounts for surrounding timing constraints, fanout loads, and interconnect effects. We study local remapping in this setting and develop a framework that couples discrete mapping search with physical implementation feedback. Timing-critical regions are isolated through bounded windows whose interfaces retain the context of the surrounding circuit. Within each window, a mixed-integer formulation jointly selects logic cuts, signal polarities, and library cells under a delay model informed by estimated locations and interconnect parasitics. A continuous relaxation filters the search space before discrete optimization produces alternative implementations with similar modeled timing and different structural choices. These implementations are reconstructed and assessed through legalization, routing-based parasitic estimation, and timing analysis. Physically validated improvements are incorporated into the design, and the updated context guides subsequent searches. The framework provides a systematic way to revisit local logic implementations while accounting for their interaction with an existing placement.

I. INTRODUCTION

Timing closure remains one of the most iterative stages in physical design. Gate sizing and buffer insertion adjust delay by modifying cell drive strength or inserting repeaters, but do not explore alternative Boolean decompositions. When the original logic decomposition is suboptimal, the existing structure can limit the timing improvements achievable through these operations. Logic restructuring complements this toolkit by replacing combinational regions with functionally equivalent implementations that may offer better timing after physical realization. Applying restructuring to an already placed design provides access to the surrounding physical context, but also requires the new implementation to remain compatible with that context.

Post-placement local restructuring raises two related challenges. First, replacing gates introduces new instances that require placement and legalization. A larger region exposes more optimization opportunities, but may disrupt more of the established placement and alter the loads seen by neighboring gates. Extraction must therefore balance optimization scope against physical disruption. Second, even a bounded region admits a combinatorial space of cut, phase, and cell choices. A mapping cost model, including one informed by physical information, may rank these choices differently from timing analysis after reconstruction, legalization, and parasitic updates.

A modeled improvement thus need not survive physical implementation. Evaluating every alternative in the placed design is costly, motivating selective candidate generation followed by physical validation.

Prior efforts connect logic optimization and physical design in several ways. Concurrent approaches interleave logic optimization with placement to account for their interaction [1]–[4]. Layout-driven synthesis also considers congestion during decomposition and mapping [5]. Physically aware technology mapping incorporates spatial information into mapping decisions: PigMAP uses primitive-gate placement to guide mapping [6], and PigMap2 extends the use of physical information in the mapping framework [7]. PigMap3 further studies incremental mapping with on-the-fly post-layout critical-path tracking [8]. CPA-Remap constructs local cones using critical paths and cell locations for physically aware remapping [9]. These approaches establish the relevance of physical context to mapping optimization. Our focus is a complementary question: how can the timing and load context of a placed design guide joint local mapping decisions while accounting for the physical effects of implementing those decisions?

Our key insight is to guide local restructuring with the timing and load context of the placed design and estimates of candidate interconnect delay, while using physical evaluation to determine candidate acceptance. Based on this insight, we propose Physically Contextualized Restructuring (PCR), which extracts bounded windows around timing-critical endpoints and jointly optimizes cut, phase, and library-cell choices. Reference timing and external pin loads retain the surrounding design context. Predicted gate positions provide wirelength and RC estimates for candidate scoring; these estimates remain fixed within each solve and can be refreshed using the selected topology. A linear programming (LP) relaxation screens windows for modeled improvement and restricts the mapping domain for integer optimization. Sparse and structurally distinct candidates are selected under a common modeled timing cap, reconstructed, and evaluated through legalization and global-routing-based timing analysis. Accepted candidates update the physical design and provide the reference context for subsequent iterations.

Our main contributions are as follows:

- A local mapping formulation that jointly models cut, phase, and cell choices within bounded windows, incorporating estimated interconnect delays while preserving shared boundary signals and the surrounding timing context (Sections III-A–III-B).
- A staged solution procedure that combines relaxation-

* Corresponding author: krzhu@fudan.edu.cn

bound screening and action-domain restriction with integer feasibility and timing optimization (Sections III-B–III-C).

- A dual-representative generation strategy that selects sparse and structurally distinct mappings under a common modeled timing cap, enabling separate physical evaluation of alternatives with similar modeled timing (Section III-C).
- An experimental evaluation showing that the complete PCR flow outperforms PigMap3 [8] with 86.7%/95.2% reductions in geometric-mean WNS/TNS violation magnitudes under position-aware postprocessing, and genetic and annealing with 48.6–49.2%/53.3–63.1% reductions under uniform postprocessing. It outperforms PigMap3 in WNS on seven of nine cases and each resynthesis baseline on six of seven cases, with one tie (Section IV).

The source code is available at <https://github.com/jzj3021/PCR>.

II. PRELIMINARIES

A. Logic Synthesis and Technology Mapping

Logic synthesis optimizes a technology-independent representation, such as an And-Inverter Graph (AIG), before mapping it to library cells. Cut-based mapping [10], [11] selects compatible implementations covering the required outputs, often from a restricted set of priority cuts [12]. Delay-oriented methods include load-aware ExactMap [13] and learned delay prediction in AiMap+ [14]. PCR uses ABC [15] to obtain legal cut, phase, and cell alternatives and their dependencies, defining the discrete domain optimized in Section III-B.

B. Linear Programming Relaxation

An LP relaxation replaces binary selections $x \in \{0, 1\}$ by $x \in [0, 1]$ while retaining the linear constraints. Let $\mathcal{M}$ be the integer feasible set, $\mathcal{P} \supseteq \mathcal{M}$ its relaxation, and $\hat{T}$ the modeled arrival-time objective. For a feasible minimization problem,

$$L_{\text{LP}} = \min_{q \in \mathcal{P}} \hat{T}(q) \leq \min_{m \in \mathcal{M}} \hat{T}(m) = J^*. \quad (1)$$

The enlarged feasible set gives a lower bound, while fractional solutions need not represent realizable mappings. PCR uses this bound to screen local mapping domains before integer candidate generation (Section III-B).

C. Placement and Timing Closure

Cell locations and interconnect geometry determine wire resistance and capacitance. Static timing analysis (STA) combines cell models with parasitics estimated from placement or global routing to compute arrival and required times. Setup slack is required time minus arrival time; negative slack indicates a violation. Worst negative slack (WNS) measures the largest setup violation, and total negative slack (TNS) sums negative endpoint slacks.

D. Post-Placement Timing Optimization

Post-placement timing optimization adjusts cell drive strengths and interconnect loads through sizing, buffering, and related transformations. Lagrangian-relaxation-based sizing [16], [17] optimizes cell sizes under timing constraints;

BufFormer learns buffer-tree topology, sizes, and locations [18]; OpenROAD’s Resizer [19] provides cell and interconnect repair operations. PCR complements these techniques by replacing local logic with equivalent mappings, using estimated positions and interconnect delays for search and physical evaluation for acceptance.

III. APPROACH

PCR consists of four stages: (1) **timing-driven window extraction**, which selects a violating endpoint and extracts a bounded logic region; (2) **mapping formulation and LP screening**, which estimates candidate positions and interconnect delays, models legal mapping choices, and screens for modeled improvement; (3) **integer mapping and reconstruction**, which uses MIP to generate concrete candidate netlists; and (4) **physical evaluation and iterative update**, which validates candidates through physical checks and global-routing-based timing evaluation before accepting a change. These stages repeat until timing is closed or the search budget is exhausted.

A. Timing-Driven Window Extraction

PCR extracts a bounded combinational window rooted at the worst-slack data endpoint, subject to limits on depth, instance count, and boundary inputs. Sequential and macro instances remain outside the window. The extracted context includes interface nets, reference STA timing, external pin loads, and the positions of existing cells, boundary sources, and external sinks. Boundary-source and external-sink positions anchor candidate position estimates. Signals that drive loads outside the window are retained as boundary outputs, preserving their functions and external connections without expanding the window to include all fanout instances.

The extracted logic defines an action graph of legal cut–phase choices, fanin dependencies, and available cell variants. Required outputs induce demands on internal nodes, and compatible actions are selected jointly to form a complete mapping.

B. Mapping Formulation and LP Screening

An action $a \in \mathcal{A}_i$ specifies a cut and its phase implementation at node i ; its cell variants specify concrete library gates, including available drive strengths. We jointly select actions and cells because a gate’s delay depends on both its own drive and its fanout loads. Let x_{ia} select an action, d_{ip} indicate demand for phase p , and r_i indicate whether either phase is demanded. The core selection constraints are

$$\sum_{a \in \mathcal{A}_i} x_{ia} = r_i, \quad d_{ip} \leq r_i \leq d_{i0} + d_{i1}, \quad \sum_a q_{iap} = d_{ip}, \quad (2)$$

where $q_{iap} = x_{ia}d_{ip}$ indicates that action a serves requested phase p . Required output phases have $d_{ip} = 1$. Each served action activates its required child phases; conversely, a non-output phase can be demanded only if a selected parent requires it. These constraints exclude unsupported internal gates. Requested phases are translated to physical gate phases according to the action’s legal phase implementation. For each such gate, $\sum_c z_{iapc} = g_{iap}$ selects exactly one compatible cell when gate-use indicator $g_{iap} = 1$.

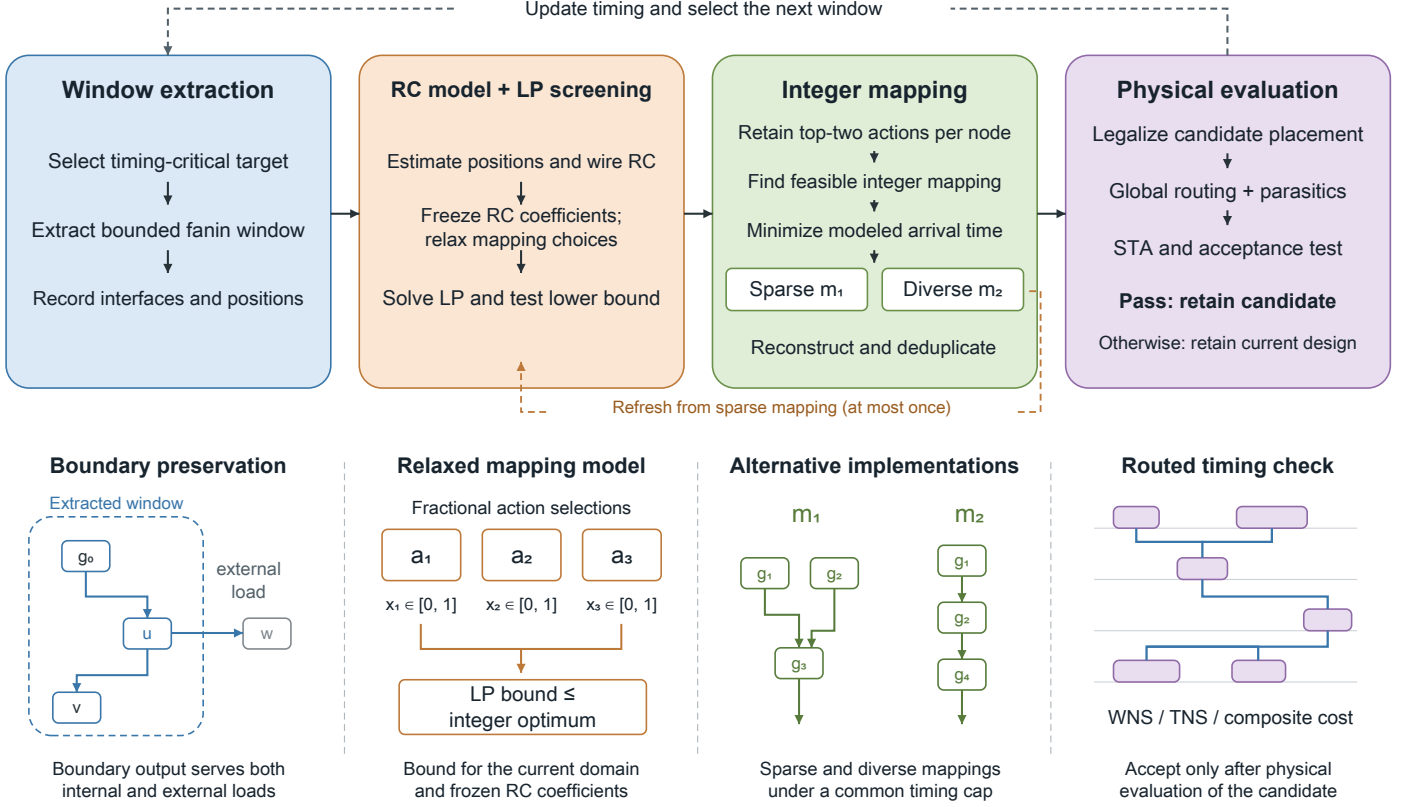


Fig. 1: Overview of PCR. Window interfaces provide fixed physical context for candidate position and RC estimation. LP screening and integer optimization use a frozen delay model to generate sparse and diverse mappings. The sparse mapping can refresh the model for one additional solve. Reconstructed candidates undergo legalization, global routing, and timing analysis before acceptance.

1) *Candidate position and interconnect estimation:* Before each solve, we estimate internal node positions from their connectivity to neighboring nodes and fixed boundary locations. Each position is iteratively moved toward a weighted centroid and projected onto a local region Ω enclosing the original window and interface positions, expanded by $2\mu\text{m}$ and clipped to the core:

$$\tilde{\mathbf{p}}_i = \Pi_{\Omega} \left(\frac{\sum_{j \in \mathcal{N}_i} w_{ij} \mathbf{p}_j}{\sum_{j \in \mathcal{N}_i} w_{ij}} \right). \quad (3)$$

Boundary sources remain fixed. The centroid includes connected neighbors, external sinks, and a weak window-center anchor; external sinks receive greater weight to prevent collapse toward the fanins. An action-specific centroid then combines its cut leaves, downstream neighbors, external sinks, and the estimated root position. These coordinates predict connectivity costs and are not placement decision variables.

For each candidate driver, a minimum spanning tree with Manhattan edge lengths approximates its internal fanout net. For clarity, action and cell indices on the tree quantities are suppressed. Let ℓ_e be an edge length, r_w, c_w the per-unit-length resistance and capacitance, and $C_e^\downarrow$ the sink and wire capacitance below the edge, excluding its own wire capacitance. The estimated wire capacitance and worst-sink Elmore delay

are

$$\begin{aligned} C_i^w &= c_w \sum_{e \in \mathcal{T}_i} \ell_e, \\ D_i^w &= \max_{s \in \mathcal{S}_i} \sum_{e \in P(i,s)} r_w \ell_e (C_e^\downarrow + \frac{1}{2} c_w \ell_e). \end{aligned} \quad (4)$$

The first solve uses possible fanouts and their maximum available input-pin capacitances for this RC estimate. If a sparse mapping is obtained, its selected topology and cell loads refresh the estimates for at most one additional solve. Both solves share the search budget. External output RC and downstream timing remain in the fixed reference context; the tree adds only internal interconnect effects.

2) *Delay model and relaxation:* We minimize modeled target arrival $\hat{T}$. With RC coefficients frozen for the current solve, the delay proxy for a selected driver variant c is

$$\begin{aligned} \hat{d}_{i,c} &= d_{i,c}^0 + \Delta d_{i,c}^{\text{RC}} + R_{i,c} \left(C_i^{\text{ext}} + \sum_j C_j^{\text{in}} z_j \right), \\ \Delta d_{i,c}^{\text{RC}} &= R_{i,c} C_i^w + D_i^w + D_{i,c}^{\text{in}}. \end{aligned} \quad (5)$$

Here j indexes fanout cell variants; the internal pin load uses their mean input-pin capacitances. For a gate directly driven by window inputs, $D_{i,c}^{\text{in}}$ is the maximum input-wire estimate $r_w L(C_{i,c}^{\text{in}} + c_w L/2)$ over its boundary-source connections of

Manhattan length L ; it is zero otherwise. Reference STA offsets calibrate the local arrival objective, while external pin loads remain fixed. The RC correction is a constant for each variant within a solve, preserving linearity. The model does not predict congestion, obstacle detours, or slew propagation. Conditional arrival constraints propagate delays through selected dependencies. Selecting a driver and its load introduces products of selection variables. Each product $w = uv$ is represented by

$$0 \leq w, \quad w \leq u, \quad w \leq v, \quad w \geq u + v - 1. \quad (6)$$

These inequalities are exact when u, v are binary, so the discrete proxy is a mixed-integer linear model.

We relax the remaining binary domains to $[0, 1]$, retaining all selection, dependency, timing, and product constraints. For fractional selections, Eq. (6) bounds their products, yielding the relaxation

$$L_{LP} = \min_{\mathcal{F}_{LP}} \hat{T} \leq \min_{\mathcal{F}_{IP}} \hat{T}, \quad \tau = \hat{T}_0 - \delta. \quad (7)$$

Here $\mathcal{F}_{IP}$ is the integer domain under the configured search limits and $\mathcal{F}_{LP}$ its relaxation. A certified bound $L_{LP} > \tau + \epsilon_{\text{cert}}$, or proven infeasibility of the threshold relaxation, rejects the modeled target; ϵ_{cert} accounts for numerical tolerance. We use the original starting-mapping score as the screening threshold ($\delta = 0$), unchanged across the two solves. The lower bound applies only to the specified candidate domain and the current frozen RC model; it does not bound timing after routing or a subsequent model update.

C. Integer Mapping and Reconstruction

Integer mapping jointly determines the gates and their connections under the current frozen delay model. We define the search domain and representative objectives below; Algorithm 1 specifies their execution order and the optional model refresh. Let m denote a model assignment with action/cell selections $\mathbf{b}(m)$ and continuous timing variables.

1) *Restrict the action domain:* Let $\bar{x}_{ia}$ be the LP value of action a at node i . We retain the two highest-ranked actions at each node (or all actions if fewer are available):

$$\begin{aligned} \mathcal{A}'_i &= \text{Top}_2\{a \in \mathcal{A}_i; \bar{x}_{ia}\}, \\ \mathcal{F}' &= \{m \in \mathcal{F}_{IP} : x_{ia} = 0 \text{ for } a \notin \mathcal{A}'_i\}. \end{aligned} \quad (8)$$

Compatible cell variants remain available. MIP jointly selects actions and cells subject to dependency and phase constraints. Infeasibility and optimality conclusions apply to the restricted domain $\mathcal{F}'$.

2) *Establish feasibility and optimize timing:* We first search for an integer assignment $m \in \mathcal{F}'$ satisfying $\hat{T}(m) \leq \tau$. Once such an incumbent is found, we solve the timing minimization problem to establish the best modeled arrival in the retained domain:

$$\hat{T}^* = \min_{m \in \mathcal{F}'} \hat{T}(m), \quad \mathcal{S} = \{m \in \mathcal{F}' : \hat{T}(m) \leq \hat{T}^* + \epsilon_T\}. \quad (9)$$

The feasibility stage can stop upon finding a threshold-feasible incumbent, avoiding a full timing optimization for windows without such an incumbent within the allotted budget. The

Algorithm 1: Candidate generation

Require: Action graph $\mathcal{G}$, physical context $\mathcal{B}$, threshold τ , tolerances $\epsilon_T, \epsilon_{\text{cert}}$

Ensure: Candidate netlist set $\mathcal{C}$

```

1:  $\mathcal{C} \leftarrow \emptyset$ ;  $m_{\text{prev}} \leftarrow \perp$ 
2: for  $k = 0, 1$  while search budget remains do
3:    $\theta_k \leftarrow \text{EstimateRC}(\mathcal{G}, \mathcal{B}, m_{\text{prev}})$ 
4:   Construct  $\mathcal{F}_{IP}, \mathcal{F}_{LP}, \hat{T}$  with frozen  $\theta_k$ .
5:    $\bar{m} \leftarrow \text{SolveLP}(\min_{\mathcal{F}_{LP}} \hat{T})$ 
6:   if  $\bar{m} = \perp$  or  $\hat{T}(\bar{m}) > \tau + \epsilon_{\text{cert}}$ : break
7:   Construct  $\mathcal{F}'$  using (8).
8:    $m_0 \leftarrow \text{FindFeasible}(\mathcal{F}' \cap \{\hat{T} \leq \tau\})$ 
9:   if  $m_0 = \perp$ : break
10:  Solve for  $\hat{T}^*$  and construct  $\mathcal{S}$  using (9).
11:  if timing optimum is unresolved: break
12:  Solve for  $m_1$  using (10).
13:  if  $m_1 = \perp$ : break
14:   $\mathcal{C}_k \leftarrow \text{ValidateAndDeduplicate}(m_1, \emptyset)$ 
15:  Solve for  $m_2$  using (10), with  $m_1$  fixed.
16:  if  $m_2 \neq \perp$  then
17:     $\mathcal{C}_k \leftarrow \text{ValidateAndDeduplicate}(m_2, \mathcal{C}_k)$ 
18:  end if
19:  if  $\mathcal{C}_k = \emptyset$ : break
20:   $\mathcal{C} \leftarrow \mathcal{C}_k$ ;  $m_{\text{prev}} \leftarrow m_1$ 
21: end for
22: return  $\mathcal{C}$ 
```

timing stage establishes a common reference for representative selection. We use $\epsilon_T = 0.1$ ps to admit alternatives with nearly equal modeled timing.

3) *Select two representatives:* Within $\mathcal{S}$, define $N(m) = \sum_{k \in \mathcal{N}} z_k(m)$, where $\mathcal{N}$ indexes non-default cell variants of the retained actions. We first minimize $N(m)$ to obtain a sparse representative m_1 . We then seek a second representative m_2 that differs most from m_1 in its action and cell selections:

$$\begin{aligned} m_1 &\in \arg \min_{m \in \mathcal{S}} (N(m), \hat{T}(m)), \\ m_2 &\in \arg \min_{m \in \mathcal{S}} (-H(m, m_1), \hat{T}(m)), \\ H(m, m_1) &= \sum_{k: b_k(m_1)=0} b_k(m) + \sum_{k: b_k(m_1)=1} (1 - b_k(m)). \end{aligned} \quad (10)$$

The sparse objective minimizes non-default cell selections, not total gate count or changes to the input netlist. The diverse objective maximizes structural difference at similar modeled timing. Both objectives use arrival time as a secondary criterion; remaining sparse ties are resolved deterministically.

4) *Reconstruct the netlists:* $\text{ValidateAndDeduplicate}(m, \mathcal{C})$ reconstructs the selected gates and connections, preserves the window interfaces, and retains compatible, realizable mappings with distinct structures or cells. It independently recomputes the original cell/load-model score for consistency checks; search uses the RC-augmented score.

TABLE I: Position-aware comparison with PigMap3 on all nine completed pairs. Both methods use detailed placement, setup timing repair, and final detailed placement.

Design	PCR (Ours)				PigMap3 [8]			
	WNS	TNS	Area	Leakage	WNS	TNS	Area	Leakage
<i>ICCAD 2024 Problem C [20]</i>								
mempool_tile_wrap	-100.90	-2994.80	21088.06	2594.1801	-8722.70	-32259792.00	21224.38	2594.2643
aes_256	0.00	0.00	24001.03	16.9181	-61.57	-356.96	24006.67	16.9176
hidden1	-141.79	-7496.61	5627.56	2.8745	-3049.58	-348824.22	5880.08	3.0498
hidden5	0.00	0.00	28068.42	26.8309	-300.52	-3544.54	28095.91	26.8359
<i>MLCAD 2025 [21]</i>								
ac97_top	-59.50	-22641.40	1157.00	83.6219	-73.65	-23640.86	1159.43	83.4392
aes_cipher_top	-22.77	-1669.02	1002.96	30.3782	-29.31	-1815.61	1003.05	30.3805
partition_m [†]	-127.15	-4891.43	1920.13	171.4317	-124.29	-4916.95	1920.22	171.4467
partition_p	-184.16	-36540.81	9269.18	612.6764	-160.88	-33286.33	9303.89	611.4790
partition_c [†]	-124.44	-1100.77	23591.66	18102.9051	-162.33	-1135.72	23593.43	18102.9473
Geo. mean*	33.10	818.82	6972.59	136.4062	248.63	16927.00	7017.36	137.2482
Ratio	1.00	1.00	1.000	1.000	7.51	20.67	1.006	1.006

WNS/TNS: ps; area: μm^2 ; leakage: μW . Bold denotes the best displayed value, including ties. Names partition_m/p/c abbreviate NV_NVDLA_partition_m/p/c. [†]One remapping update accepted; the other seven cases retain their input mappings. *Timing geometric means use $G = [\prod_{i=1}^n (|x_i| + 1)]^{1/n} - 1$ with $n = 9$; area/leakage use ordinary geometric means. Ratio is $G_{\text{method}}/G_{\text{PCR}}$; values above one favor the PCR flow. Lower aggregates are better.

In Algorithm 1, $\perp$ denotes failure to meet a solve’s completion criterion: feasibility for FindFeasible or optimality within solver tolerances for optimization. A timeout does not prove infeasibility. An unresolved diverse representative leaves the sparse candidate available; an incomplete model-refresh round retains the previous completed candidate set.

D. Physical Evaluation and Iterative Update

Predicted coordinates serve only the delay model. Each reconstructed candidate is applied to a separate copy of the current placed design, where actual placement and legalization determine its physical locations. Detailed placement legalizes the modified region, followed by placement checking, global routing, parasitic estimation, and STA. The candidate and reference are evaluated using the same timing constraints and physical settings.

A candidate is accepted when WNS and TNS do not degrade beyond tolerance, at least one improves, and the composite cost decreases. The tolerances and cost function are specified in Section IV-A. If the candidate passes, we retain its design, recompute the search context, and restart endpoint selection from the worst slack. Otherwise, we retain the previous design, try any remaining representative, and then advance to the next endpoint. Iteration stops when setup timing is closed, the window limit is reached, or the wall-time budget expires.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. Experimental Setup

Experiments run on a Linux cluster with Intel Xeon processors, using single-threaded workers. Both benchmark suites use ASAP7 libraries [22]. OpenROAD [19] provides the physical-design backend for placement, timing repair, global routing, parasitic estimation, and static timing analysis. We compare PCR with OpenROAD’s genetic-algorithm and simulated-annealing based restructuring methods (resynth_genetic

and resynth_annealing), and with PigMap3 [8]. All methods use the same initial checkpoint, libraries, timing constraints, and final evaluation settings for each design.

We evaluate benchmarks from the 2024 ICCAD Contest Problem C [20] and the 2025 MLCAD Contest [21]. Table I lists all nine completed position-aware pairs with PigMap3 [8]; Table II lists all seven completed uniform-flow comparisons with annealing and genetic. Each table uses a common case set for its methods and reports WNS, TNS, standard-cell area, and leakage power.

PCR uses extraction limits of depth six, 60 instances, and 40 boundary inputs, with an endpoint pool of 256. Each run attempts at most 20 windows within $\max(1800, \lceil 2t_{\text{ref}} + 600 \rceil)$ seconds, where t_{ref} is reference evaluation time. The LP/MIP selection rules and timing tolerances are specified in Section III. Resynthesis and physical-stage timeouts are 7200 and 14400 s, respectively; search budgets are method-specific.

During local restructuring, routed candidate acceptance allows at most 0.001 ps degradation in WNS/TNS, requires improvement exceeding 0.001 ps in at least one, and requires a decrease exceeding 10^{-6} in

$$C = P_{\text{leak}} + 10|\min(\text{TNS}, 0)| + 20V_{\text{slew}} + 20V_{\text{cap}}, \quad (11)$$

with leakage, TNS, slew violation, and capacitance violation expressed in μW , ns, ns, and ff.

B. Experimental Flow

We organize comparisons by matched physical postprocessing. PCR and PigMap3 [8] are compared in the *position-aware flow*: detailed placement, setup timing repair, and final detailed placement. Annealing and genetic are compared with PCR in the *uniform flow*: global and detailed placement both before and after timing repair. These settings assess placement-preserving optimization and optimization followed by common global re-placement, respectively. For each method,

TABLE II: Uniform-flow comparison on all seven cases completed by all three methods. Every method uses global and detailed placement before and after setup timing repair.

Design	PCR (Ours)				Annealing				Genetic			
	WNS	TNS	Area	Leakage	WNS	TNS	Area	Leakage	WNS	TNS	Area	Leakage
<i>ICCAD 2024 Problem C [20]</i>												
partition_m	-236.59	-10784.62	2759.48	2.7394	-438.20	-21553.12	2073.74	1.9454	-485.89	-23138.89	2302.42	2.1421
partition_p	-172.12	-21405.49	10850.60	6.6562	-391.17	-100520.52	10587.47	6.4325	-375.68	-98190.02	10675.18	6.5484
aes_256	-427.70	-9144.16	24001.04	16.9181	-427.70	-9144.16	24001.04	16.9181	-427.70	-9144.16	24001.04	16.9181
hidden1	-73.06	-2843.29	5627.71	2.8747	-112.18	-4764.22	5682.15	2.9453	-125.23	-1298.82	5706.71	2.9664
hidden5	-156.44	-4817.11	28104.82	26.8765	-157.45	-4138.66	28104.82	26.8765	-157.45	-4138.66	28104.82	26.8765
<i>MLCAD 2025 [21]</i>												
ac97_top	-65.50	-24721.82	1156.16	83.4442	-162.46	-142236.77	1271.77	66.7008	-159.43	-123481.26	1291.86	72.3576
aes_cipher_top	-41.80	-2806.58	1009.81	30.4120	-301.08	-39003.28	1292.66	35.4963	-236.81	-30209.31	1266.97	31.8558
Geo. mean*	127.71	7949.77	5393.12	12.9319	251.47	21563.35	5425.30	12.1761	248.39	17038.68	5513.40	12.3416
Ratio	1.00	1.00	1.000	1.000	1.97	2.71	1.006	0.942	1.94	2.14	1.022	0.954

WNS/TNS: ps; area: μm^2 ; leakage: μW . Bold denotes the best displayed value, including ties. Names partition_m/p/c abbreviate NV_NVDLA_partition_m/p/c. All seven cases retain their input mappings during remapping. *Timing geometric means use $G = [\prod_{i=1}^n (|x_i| + 1)]^{1/n} - 1$ with $n = 7$; area/leakage use ordinary geometric means. Ratio is $G_{\text{method}}/G_{\text{PCR}}$; values above one favor the PCR flow. Lower aggregates are better.

the two flows reuse the same mapping output. Repair uses `repair_timing -setup`, allowing sizing, buffering, pin swapping, and cloning. Final checkpoints undergo legalization and global-routing-based timing evaluation.

C. Results Analysis

Each table reports geometric means and ratios over its own completed case set. Timing aggregates use add-one smoothing to include zero violations. The two tables have different case sets and physical flows, so their aggregate values are not directly comparable.

1) *Position-Aware Comparison with PigMap3*: The PCR flow achieves better WNS on seven of nine cases and better TNS on eight. It closes timing on `aes_256` and `hidden5`, and gives lower area on all nine cases. The large aggregate timing advantage reflects both these zero-violation results and large PigMap3 violations on `mempool_tile_wrap` and `hidden1`; its magnitude depends on the smoothing convention.

Only MLCAD `partition_m` and `partition_c`, marked in Table I, accept a remapping update. Before timing repair, these updates improve WNS by 48.61 and 1976.43 ps relative to their respective initial designs. After the complete flow, `partition_c` improves WNS/TNS over PigMap3 by 37.90/34.95 ps, with slightly smaller area and leakage. On `partition_m`, TNS violation decreases by 0.52%, while WNS worsens by 2.86 ps. The remaining seven cases retain their input mappings, so the table’s overall advantage measures complete-flow outcomes rather than isolated remapping gains.

2) *Uniform Comparison with Annealing and Genetic*: Across seven completed cases, the PCR flow reduces geometric-mean WNS and TNS violation magnitudes by 48.6–49.2% and 53.3–63.1%, respectively, relative to annealing and genetic. It has the best WNS on all seven cases, including a tie on `aes_256`, and the best TNS on five, also including that tie. Area and leakage exhibit design-dependent tradeoffs, as shown in Table II.

V. CONCLUSION

We presented PCR, a physically contextualized local restructuring method for post-placement timing optimization. PCR extracts bounded windows around timing-critical endpoints while preserving their interfaces to the surrounding design. Within each window, PCR predicts gate positions and interconnect RC to jointly optimize cut, phase, and cell choices under a delay model frozen for each solve. The selected topology can refresh this model once; an LP relaxation screens for modeled improvement and guides action-domain restriction, followed by integer optimization to select sparse and structurally diverse candidate mappings. Candidates are reconstructed, legalized, and evaluated using global-routing parasitics before acceptance. Accepted updates guide subsequent window selection, enabling iterative optimization within a fixed computation budget. Experimental results show that the complete PCR flow outperforms PigMap3 [8] with 86.7% and 95.2% reductions in geometric-mean WNS and TNS violation magnitudes under the position-aware flow, and genetic and annealing with reductions of 48.6–49.2% and 53.3–63.1% under the uniform flow. It outperforms PigMap3 in WNS on seven of nine cases and each resynthesis baseline on six of seven cases, with one tie.

REFERENCES

- [1] Y.-L. Jiang, S. Krstic, K.-T. Cheng, and M. Marek-Sadowska, "Post-layout logic restructuring for performance optimization," in *Proceedings of the 34th Design Automation Conference (DAC)*, 1997, pp. 662–667.
- [2] W. Gosti, S. P. Khatri, and A. L. Sangiovanni-Vincentelli, "Addressing the timing closure problem by integrating logic optimization and placement," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2001.
- [3] J. Lou, W. Chen, and M. Pedram, "Concurrent logic restructuring and placement for timing closure," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 1999, pp. 31–36.
- [4] L. Stok, A. J. Brodnax, R. L. Bryles, D. L. Cerny, S.-K. Huang, and D. S. Kung, "Cycle time optimization by timing driven placement with simultaneous netlist transformations," in *Proceedings of the 38th Design Automation Conference (DAC)*, 2001.
- [5] T. Kutzschebauch and L. Stok, "Congestion aware layout driven logic synthesis," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2001.
- [6] H. Pan, C. Lan, Y. Liu, Z. Wang, L. Shang, X. Zeng, F. Yang, and K. Zhu, "Physically aware synthesis revisited: Guiding technology mapping with primitive logic gate placement," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3676536.3676676>
- [7] C. Lan, H. Pan, Z. Wang, X. Zeng, F. Yang, and K. Zhu, "Pigmap2: A physical information guided technology mapping framework," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2025.
- [8] H. Pan, C. Lan, Z. Wang, X. Zeng, F. Yang, and K. Zhu, "Pigmap3: A physically aware incremental mapping framework with on-the-fly post-layout critical path tracking," in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2026, pp. 815–821.
- [9] M. He, P. Huang, Z. Zhao, and P. Bian, "CPA-Remap: Critical-Path-Based Physically Aware Remapping Framework for Timing Optimization," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. Richardson, TX, USA: IEEE, Nov. 2025, pp. 393–400. [Online]. Available: <https://ieeexplore.ieee.org/document/11310850/>
- [10] K. Keutzer, "Dagon: Technology binding and local optimization by dag matching," in *24th ACM/IEEE Design Automation Conference*, 1987, pp. 341–347.
- [11] S. Chatterjee, A. Mishchenko, R. Brayton, X. Wang, and T. Kam, "Reducing structural bias in technology mapping," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 12, pp. 2894–2903, 2006.
- [12] A. Mishchenko, S. Cho, S. Chatterjee, and R. Brayton, "Combinational and sequential mapping with priority cuts," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2007, pp. 354–361.
- [13] Z. Xie, L. Liu, T. Liu, and E. F. Young, "ExactMap: Enhancing Delay Optimization in Parallel ASIC Technology Mapping," in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. Munich, Germany: IEEE, Oct. 2025, pp. 1–9. [Online]. Available: <https://ieeexplore.ieee.org/document/11240698/>
- [14] J. Liu and Q. Zhao, "AiMap+: Guiding Technology Mapping for ASICs via Learning Delay Prediction," *Electronics*, vol. 13, no. 18, p. 3614, Sep. 2024. [Online]. Available: <https://www.mdpi.com/2079-9292/13/18/3614>
- [15] A. Mishchenko, S. Chatterjee, and R. Brayton, "Dag-aware aig rewriting: a fresh look at combinational logic synthesis," in *2006 43rd ACM/IEEE Design Automation Conference*, 2006, pp. 532–535.
- [16] V. S. Livramento, C. Guth, J. L. Güntzel, and M. O. Johann, "Fast and efficient lagrangian relaxation-based discrete gate sizing," in *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2013, pp. 1855–1860.
- [17] G. Flach, T. Reimann, G. Posser, M. Johann, and R. Reis, "Effective method for simultaneous gate sizing and v_{th} assignment using lagrangian relaxation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 4, pp. 546–557, 2014.
- [18] R. Liang, S. Nath, A. Rajaram, J. Hu, and H. Ren, "BufFormer: A Generative ML Framework for Scalable Buffering," in *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. Tokyo Japan: ACM, Jan. 2023, pp. 264–270. [Online]. Available: <https://dl.acm.org/doi/10.1145/3566097.3567900>
- [19] T. Ajayi, D. Blaauw, T. Chan, C. Cheng, V. Chhabria, D. Choo, M. Coltella, S. Dobre, R. Dreslinski, M. Fogaça *et al.*, "Openroad: Toward a self-driving, open-source digital layout implementation tool chain," *Proc. GOMACTECH*, pp. 1105–1110, 2019.
- [20] B.-Y. Wu, R. Liang, G. Pradipta, A. Agnesina, H. Ren, and V. A. Chhabria, "Invited paper: 2024 ICCAD CAD contest problem c: Scalable logic gate sizing using ML techniques and GPU acceleration," in *2024 ACM/IEEE International Conference on Computer Aided Design (ICCAD)*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/11126197/>
- [21] V. Gopalakrishnan, A. Dey, R. Liang, Y. Zhang, H. Ren, and V. A. Chhabria, "Invited Paper: MLCAD 2025 Contest on ReSynthAI: Physical-aware Logic Resynthesis using AI," in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*. Santa Cruz, CA, USA: IEEE, Sep. 2025, pp. 1–7. [Online]. Available: <https://ieeexplore.ieee.org/document/11189135/>
- [22] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "Asap7: A 7-nm finfet predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002626921630026X>